

PHYNETPY: a Python framework for phylogenetic network analysis and inference

Mark Kessler*

Luay Nakhleh

Department of Computer Science, Rice University

Abstract

Motivation. Phylogenetic network methods have accumulated faster than the software infrastructure to support them. PhyloNet established a body of network-analysis methods and the computational ecosystem around them, and much of the field’s methodological work now assumes that body of methods exists. New methods, however, are typically built as standalone programs, each supplying its own network representation, search machinery and input handling, and most are not reachable from the Python environment where comparative and statistical analysis is now usually done.

Results. We present PHYNETPY, a Python library for phylogenetic network inference and analysis. Inference is expressed as two verbs, `infer` and `score`, parameterised on three independent axes: the data observed, the generative model, and the statistical criterion. A method is a cell in that product rather than a command name, and dispatch runs through a registry that doubles as a machine-readable statement of which combinations are defined, which are unimplemented, and which are meaningless. Version 1.0.0 provides seven inference methods, six of them reimplementations of published PhyloNet analyses; a reusable rooted-DAG network type with ploidy and typed branch-length units; nine network dissimilarity measures and a separate reticulation-specific comparison module; coalescent, sequence and biallelic-marker simulation; and MCMC convergence diagnostics with Tracer-compatible output. The library holds 1,130 tests, and a harness cross-validates its multispecies-network-coalescent density and Felsenstein likelihood against PhyloNet’s own implementations across 14 adversarial cases, agreeing to 10^{-5} . On matched pseudo-likelihood inference from identical gene trees, PHYNETPY runs faster than PhyloNet by a margin that grows with dataset size, but recovers the generating topology about half as often across 30 trials; we report that discrepancy and the diagnostic work it still needs rather than a net verdict.

Availability. `pip install phynetpy`. Source, documentation and the scripts reproducing every figure and table here are at <https://github.com/NakhlehLab/PhyNetPy>, under the MIT license.

1 Introduction

Species histories are not always trees. Hybridization, introgression and allopolyploidy move genetic material between diverging lineages, and the result is a reticulate history that a tree cannot express [1]. Inferring such histories is harder than inferring trees for two reasons that compound: the space of networks is larger and less well behaved than the space of trees, and the same gene-tree discordance that hybridization produces is also produced by incomplete lineage sorting [2].

*Corresponding author: mak17@rice.edu

Distinguishing the two requires a model in which both occur, which is what the multispecies network coalescent provides [3].

Over roughly fifteen years PhyloNet [4, 5] assembled the methods this implies: parsimony, maximum likelihood, pseudo-likelihood, and Bayesian inference, from gene trees, from sequence alignments, and from biallelic markers, with extensions to polyploid complexes. Its importance to the field is not in dispute and this paper does not argue otherwise. A researcher today who wants to infer a phylogenetic network from gene trees reaches for PhyloNet, and should.

What has changed is the surrounding computational environment. Three specific frictions motivated this work.

First, a PhyloNet analysis is specified in a NEXUS file containing a command block and run through the JVM. That design is well suited to running an analysis and poorly suited to embedding one: a sweep over conditions means generating files, and consuming the result means parsing text. Comparative and statistical analysis in phylogenetics now largely happens in Python, alongside NumPy [6] and SciPy [7], and the tree-oriented libraries that serve that environment – DendroPy [8], ETE [9], Biopython [10] – have no network counterpart.

Second, a method developer starting a new network method starts from very little. A network representation, traversal and cluster machinery, topology moves, a search driver, parameter optimisation, input parsing and convergence diagnostics are all prerequisites to testing a new scoring criterion, and none of them are what the new method is about. The predictable outcome is that each new method ships as a separate program with its own representation, and that comparing two methods means reconciling two file formats.

Third, the organisation of a method set as a list of command names hides its structure. Most of those commands vary along the same three dimensions – the data, the assumed generative process, the optimality criterion – and a list does not say which combinations exist, which are unimplemented, and which are not meaningful at all. A user cannot tell the difference between a feature that is missing and a request that does not make sense.

PHYNETPY addresses these. It is a Python library in which the network is a reusable data structure, inference is two verbs over three typed axes, and the set of available methods is queryable at runtime. Its inference methods are, for the most part, reimplementations of PhyloNet’s, which is deliberate: the point is not new statistics but a foundation on which new statistics can be built and against which existing results can be checked.

Contributions.

- A network representation and analysis layer usable independently of any inference method: a rooted DAG with ploidy, typed branch-length units, nine dissimilarity measures, blob and cluster decomposition, and MUL-tree conversion (Sections 3 and 4).
- An inference interface of two verbs over the product of data, model and criterion, with dispatch through a registry that distinguishes undefined, unimplemented and ill-typed requests, and that serves as the extension point for contributed methods (Section 3).
- Seven implemented inference methods, six corresponding to published PhyloNet analyses, with the gaps relative to PhyloNet stated explicitly (Section 4).
- Numerical cross-validation against PhyloNet’s own implementations of the multispecies-network-coalescent density and the Felsenstein likelihood, across 14 adversarial cases (Section 5).

- A replicated head-to-head runtime and accuracy comparison on identical input, reporting a runtime advantage and an accuracy deficit, with the accuracy result left open rather than explained away (Section 5).
- A migration account for existing PhyloNet users, including what does not carry over (Section 6).

2 Background

This section fixes terminology and names the four criterion families the library implements. Readers familiar with network inference can skip to Section 3.

2.1 Networks

A rooted phylogenetic network is a rooted directed acyclic graph whose leaves are labelled by sampled taxa. Nodes of in-degree one are tree nodes and represent ordinary divergence; nodes of in-degree two or more are reticulation nodes and represent an event in which a lineage derives material from more than one parent. Edges entering a reticulation carry inheritance probabilities γ summing to one, giving the proportion of loci descending through each parent.

A network with no reticulation nodes is a tree, which is why PHYNETPY does not give trees a separate type. Two derived notions recur. A *displayed tree* is obtained by retaining one incoming edge at each reticulation and suppressing the rest; a network on r reticulations displays up to 2^r trees, each with a probability given by the product of the retained γ values. A *blob* is a biconnected component, and contracting each blob to a node yields the *tree of blobs*, which captures the part of the history that is unambiguously tree-like. Network *level* is the largest number of reticulations in any single blob, and bounds how entangled the reticulate structure is.

Branch lengths are measured on more than one scale. Sequence-based methods work in expected substitutions per site; coalescent methods work in coalescent units, related through the population-size parameter $\theta = 4N\mu$. The two are not interchangeable, and a network carrying lengths on one scale will produce plausible but wrong answers if read on the other.

2.2 Criteria

Parsimony. Minimising deep coalescences (MDC) counts the extra lineages implied by embedding observed gene trees in a candidate species history and prefers the history requiring fewest [11]. It needs only gene-tree topologies, is cheap, and assumes no probabilistic model. Under polyploidy the criterion is applied to a multiply-labelled tree, in which a polyploid taxon appears once per subgenome [12].

Likelihood. Under the multispecies network coalescent, the probability of a gene tree given a network is obtained by summing over the ways lineages can be routed through reticulations and coalesce within branches [13]. This uses all the information in the data and is the most expensive of the criteria, since the sum grows quickly with taxa and reticulations.

Pseudo-likelihood. Replacing the full likelihood with a product over subsets of taxa – in practice rooted triplets – yields a composite objective that is far cheaper and retains enough signal to rank topologies [14]. The cost is that the product is not a normalised probability of the data, which is why it cannot serve as a posterior target.

Bayesian inference. Sampling networks from a posterior requires proposals that change dimension, since adding or removing a reticulation changes the parameter count; reversible-jump MCMC supplies the machinery [15, 16]. Samplers exist for gene trees as data, for sequence alignments with gene trees treated as latent variables [17], and for biallelic markers, where the marker likelihood is computed exactly without gene trees [18, 19].

The grouping above is the reason PHYNETPY’s interface has the shape it does. Across these four families the data type, the assumed generative process and the objective vary independently, so the natural parameterisation of the method set is a product of three axes rather than a list of names.

3 Software architecture

Figure 1 shows how PHYNETPY is put together. The organising decision is that no analysis owns the network. A network is a data structure that the library provides; scoring criteria, generative models, topology moves and search drivers are separate things that act on it. Almost everything else in this section follows from that separation.

3.1 The network as a reusable object

Network is a rooted directed acyclic graph over **Node** and **Edge**, with **Branch** carrying length, inheritance probability and tag. Trees are not a separate type: a tree is a network with no reticulation nodes, and the same code paths serve both. Gene trees supplied as input are **Network** instances, which is why a gene tree and an inferred species network can be passed to the same distance function.

Reticulation in-degree is not capped at two. **Network.get_parents** imposes no limit and **add_edges** does not check parent counts, so a node representing an unresolved or aggregated reticulation with three or more parents is representable. Some analyses do assume the binary case – **reticulation_tripartitions** requires exactly two parents, for instance – so the generality lives in the representation rather than throughout the library.

The operations on **Network** are the ones network methods keep needing: topological ordering, breadth- and depth-first traversal, most recent common ancestors, leaf-descendant sets for all nodes at once, induced subnetworks, acyclicity checking, and copying. Ploidy is first class through **subgenome_count** and the corresponding edge queries, and conversion to a multiply-labelled (MUL) tree is implemented because the allopolyploid parsimony criterion is defined on that representation.

Two design choices in this layer are worth stating because they change what callers can get wrong. Branch lengths carry an explicit unit – **SUBSTITUTIONS_PER_SITE**, **COALESCENT_2N**, or **UNSPECIFIED** – and **convert_network_branch_lengths** refuses to convert when either side is unspecified rather than guessing. Coalescent methods and sequence methods read branch lengths on different scales, and mixing them produces plausible numbers rather than an error, so the unit is tracked on the network instead of being left to the user’s discipline. Second, the node and edge containers are compiled (**graph_core_cy**) and maintain indices for incidence and name lookup, because network search evaluates large numbers of candidate topologies and the cost of a single edge insertion sets the ceiling on tractable problem size.

3.2 Three axes and three verbs

A phylogenetic network method is usually presented as a named procedure. Seen from the software side, most of those names vary along three independent dimensions: the data observed, the gen-

erative process assumed, and the statistical objective optimised. `PHYNETPY` makes those three dimensions explicit arguments and reduces the interface to three verbs.

`infer` searches for a network. `score` evaluates one. `simulate` runs the axes backwards, taking a model and a network and returning a data object. The axes are:

- **Data** — `GeneTrees`, `Alignment`, `BiallelicMarkers`. Each carries the species-to-allele mapping, and each knows its own type, so a caller never declares what kind of data it is passing.
- **Model** — `MSC`, the multispecies network coalescent with a population parameter $\theta = 4N\mu$, and `Allopolyploid`, which takes a subgenome map. The generative process used to be implicit in which scorer a user selected.
- **Criterion** — `MDC` (minimising deep coalescences), `Likelihood`, `PseudoLikelihood`, and `Bayesian`.

`Bayesian` wraps an objective rather than sitting beside the likelihoods: `Bayesian(objective=Likelihood())`. Sampling a posterior is a mode of working with a likelihood, not a fourth thing to optimise, and expressing it that way means `Bayesian` inherits the data types its objective accepts instead of restating them.

The practical consequence is that changing method means changing an argument. The same call that maximises a pseudo-likelihood samples a posterior if the criterion changes, and `infer` returns the same type either way: `InferenceResult` carries `.best`, `.score`, `.trace`, and `.posterior`, the last populated only for Bayesian runs. Attribute access falls through to `.raw`, the engine's native result, so per-method richness – log writers, information criteria, reticulation posteriors – remains reachable without the wrapper having to enumerate it.

3.3 Dispatch as a validity matrix

Whether a run makes sense depends on the whole triple, not on any one axis. A parsimony criterion over a sequence alignment is not a missing feature; it is a category error, since minimising deep coalescences is defined over gene-tree topologies. Maximum parsimony from gene trees under the `MSC`, by contrast, is perfectly well defined and simply has no implementation here yet. A criterion that needs branch lengths applied to gene trees that lack them is a third, different problem.

`resolve` in `_registry.py` separates these three cases and raises accordingly: `TypeError` when the criterion is not defined on that data type, `ValueError` when the data lack branch lengths the criterion requires, and `NotImplementedError` when the combination is meaningful but unbuilt. Conflating them is what makes an unsupported request hard to act on: a user cannot tell whether to supply different data, wait for a release, or abandon the idea.

Because dispatch is a registry keyed on the triple, the set of available methods is introspectable. `validity_matrix()` tabulates it, and the table is built from the registry rather than maintained alongside it, so it cannot fall out of date. `registered_cells()` lists every implemented combination; Table 1 is generated from that call.

3.4 Extension

Adding a method means registering a cell:

```
@register(GeneTrees, MSC, MyCriterion)
class MyEngine(Engine):
    method = "MyMethod"
```

```
def infer(self, data): ...
def score(self, network, data, optimize=False): ...
```

What an engine author does not write is the rest of the run. The search drivers (`HillClimbing`, `SimulatedAnnealing`, and the `ProposalKernel` interface), the ten topology-move classes, the Brent coordinate-ascent optimiser for branch lengths and inheritance probabilities, the I/O layer and the chain diagnostics are shared. This is the claim in the paper’s title that rests on design rather than measurement, so it is worth being precise about the evidence: the registry is exercised by all seven of the library’s own methods, which is a weaker statement than demonstrating that an outside contributor found it sufficient.

One cross-cutting option illustrates the difference between a constraint and a hint. A starting phylogeny can be supplied as `Start(net)`, a free starting point, or as `Start(net, StartMode.AUGMENT)`, which requires the result to contain that network. The second is enforced by a cluster-containment check in the accept path rather than approximated by withholding moves, which is possible because adding reticulations to a network preserves its clusters. Engines whose proposal kernels cannot honour it – `MCMC_SEQ` and the allopolyploid search – reject the request explicitly instead of accepting it and quietly doing something else.

3.5 Compiled kernels

Four Cython modules are compiled at install time. `graph_core_cy` provides the node and edge containers; `gt_msc_cy` and `mpl_engine_cy` accelerate the gene-tree coalescent likelihood and the pseudo-likelihood dynamic program. The fourth, `seq_engine_cy`, is compiled but not yet called from `_seq_likelihood.py`.

None of these are part of the public interface, and the graph core is a hard requirement rather than an optional accelerator: `Network` imports it at module load and raises with build instructions if it is absent. That removes a class of bug in which a user unknowingly runs a slower parallel implementation, at the cost of requiring a C compiler when installing from source. Section 5 reports what the compiled pseudo-likelihood kernel is worth against the pure-Python path that remains in the repository as a reference.

4 What version 1.0.0 does

4.1 Inference

Table 1 lists the analyses available, generated from `registered_cells()` so that it matches the release it describes. Seven cells are implemented. Six correspond to PhyloNet commands and are reimplementations of published methods; the seventh, maximum-parsimony inference under polyploidy, is a new implementation of the method PhyloNet exposes as `InferNetwork_MP_Allopp` and differs from it in its proposal kernel.

Three points about that table matter more than its length.

The first is what is *not* in it. Maximum parsimony from gene trees under diploid hybridization – PhyloNet’s `InferNetwork_MP` – is a legal cell with no engine behind it. The only parsimony code in `PHYNETPY` is defined on multiply-labelled trees for the allopolyploid model, so the diploid case is genuinely absent rather than partially present. The biallelic pseudo-likelihood is absent for the same kind of reason: the marker code computes the full likelihood only. Both are scheduled for

Engine	Data	Model	Criterion	Problem addressed	PhyloNet command	Status
MCMC_GT	Gene trees	MSC	Bayesian	Posterior over networks given gene trees [20]	MCMC_GT	Reimpl.
InferNetwork_ML	Gene trees	MSC	Likelihood	Network topology and parameters by maximum likelihood [13]	InferNetwork_ML	Reimpl.
InferNetwork_MPL	Gene trees	MSC	Pseudo-lik.	Network topology from gene-tree triplet frequencies [14]	InferNetwork_MPL	Reimpl.
MCMC_SEQ	Alignment	MSC	Bayesian	Joint posterior over networks and gene trees from sequences [17]	MCMC_SEQ	Reimpl.
MCMC_BiMarkers	Markers	MSC	Bayesian	Posterior over networks from biallelic markers [19]	MCMC_BiMarkers	Reimpl.
MLE_BiMarkers	Markers	MSC	Likelihood	Exact marker likelihood of a given network [18, 19]	MLE_BiMarkers	Reimpl., scoring only
MP_Allopp	Gene trees	Allopolyploid	Parsimony	Parsimonious network under polyploidy, minimising deep coalescences [11, 12]	InferNetwork_MP_Allopp	New impl.
<i>Cells that are well defined but carry no engine</i>						
—	Gene trees	MSC	Parsimony	<i>raises NotImplementedError</i>	InferNetwork_MP	Sched. 1.0.0
—	Markers	MSC	Pseudo-lik.	<i>raises NotImplementedError</i>	MLE_BiMarkers -pseudo	Sched. 1.0.0
—	Alignment	MSC	Likelihood	<i>raises NotImplementedError</i>	--	Not planned

Table 1: Analyses in PHYNETPY 1.0.0. Rows are generated from the dispatch registry. “Reimplementation” means the method was published elsewhere and implemented here from its description; the **MP_Allopp** row is marked as a new implementation because its search uses a single ploidy-preserving move rather than the six moves of the original. The lower block lists combinations that are well defined but carry no engine, and therefore raise **NotImplementedError** rather than failing obscurely.

1.0.0.¹

The second is a limitation inside a row that is otherwise marked available. **MLE_BiMarkers** can score a given network but cannot search: its **infer** path raises unconditionally. A reader comparing feature lists would otherwise reasonably assume that maximum-likelihood network inference from markers is available.

The third concerns **Bayesian(objective=PseudoLikelihood())**, which is refused. A triplet pseudo-likelihood is not a normalised probability of the data, so using it as a Metropolis–Hastings target does not yield a calibrated posterior. The combination is mechanically easy to allow and would produce numbers; it is rejected because those numbers would not mean what a reader of the output would take them to mean.

Search behaviour is controlled by presets rather than by combinations of booleans. Four are provided: **default**, **fast**, **accurate**, and **phylonet**. The last exists to reproduce PhyloNet’s optimise-everything-per-topology behaviour, which is what makes cross-checking against it meaningful; individual flags override the preset.

4.2 Network analysis

The analysis layer is usable without running any inference, and is where a researcher comparing methods or summarising results spends time.

¹[TODO: verify at 1.0.0. At the time of writing these two cells raise **NotImplementedError**; the manuscript must be updated with measured behaviour once they are built, or this claim removed.]

Nine dissimilarity measures are implemented: the μ (path-multiplicity) distance [21], hardwired and softwired cluster distances, Robinson–Foulds over hardwired clusters, tripartition distance, displayed-tree distance, average and weighted average path distance [22], and a pairwise leaf distance; **Network** additionally provides a rooted triplet distance. The cluster, tripartition and displayed-tree measures accept a `normalize` argument returning a value in $[0, 1]$.

A separate module compares networks by the reticulations they assert rather than by overall topology, through reticulation tripartitions, a dissimilarity built on them, and precision and recall against a reference network. Whether an inferred network recovers the *right hybridizations* is the question a biologist usually has, and a whole-network distance answers it only indirectly.

Decomposition covers displayed trees with their probabilities, the dominant tree, hardwired and softwired clusters, biconnected components (blobs) and the tree of blobs, network level, bridges and articulation points, and induced subnetworks by taxon set.

4.3 Simulation

`simulate` is the generative inverse of `infer` and returns a data-axis object, so a recovery check is two calls with nothing in between:

```
sim = simulate(MSC(theta=0.02), taxa=6, n=200)
recovered = infer(sim, criterion=PseudoLikelihood())
```

Gene genealogies are simulated backwards in time within network branches under the multi-species network coalescent, with lineages routed at reticulations according to inheritance probabilities. Sequence evolution runs a continuous-time Markov chain down each gene tree under JC69, HKY85 or GTR, single- or multi-locus. Biallelic markers are simulated site by site, sampling an independent genealogy per site. Species trees come from Yule or constant-rate birth–death processes; networks are produced by drawing a tree and grafting reticulations onto it.

Two limits are worth stating. Simulation under **Allopolyploid** raises **NotImplementedError** – only **MSC** has simulators. And because reticulations are grafted onto a separately drawn tree, the network simulator is not a birth–death–hybridization process in which hybridization and speciation occur jointly, so it should not be used to study the distribution of network shapes such a process induces.

One asymmetry catches users. The simulators write branch lengths in expected substitutions per site while the gene-tree criteria read coalescent units, so a round trip recovers the topology but not the lengths. The branch-length unit tags described in Section 3 make this visible rather than silent.

4.4 Data, I/O, and diagnostics

Extended Newick, NEXUS, FASTA and VCF are supported for both reading and writing. The extended Newick reader detects dialects and converts between them, which matters because no single interchange convention is universal across network software. PHYLIP is not supported.

Four alphabets are built in (DNA, RNA, protein, codon). Eight nucleotide substitution models are provided – JC, K80, F81, HKY, TN93, K81, SYM, GTR – each parameterised by the free parameters its literature defines rather than by a full exchangeability vector.

For posterior analysis, **PHYNETPY** computes effective sample size, autocorrelation time, standard error of the mean, highest-posterior-density intervals, the Geweke statistic [23], and the Gelman–Rubin $\hat{R}$ across chains [24]. Traces are written in Tracer log format [25] and tree samples in NEXUS, and Tracer logs can be read back, so existing tooling applies to **PHYNETPY** output

without conversion scripts. `reticulation_sweep` fits a range of reticulation counts and reports information criteria, which addresses the standing problem that a more reticulate network almost always fits the data better.

5 Validation and performance

Two questions are often run together and answered with the same number. Does PHYNETPY compute what it claims to compute? And how fast is it? The first is a correctness question with a right answer; the second is a measurement that depends on hardware, data, and what each implementation was asked to do. This section keeps them apart.

All measurements below were taken on the platform recorded in Section 10 and are regenerated by scripts in the repository; no figure in this section was entered by hand.

5.1 Agreement with PhyloNet

The strongest correctness evidence available for a reimplement is numerical agreement with the implementation it reimplements, on inputs chosen to be difficult. PHYNETPY ships a harness that compiles a Java driver invoking PhyloNet’s own classes – `GeneTreeBrSpeciesNetDistribution` for the multispecies-network-coalescent density, and its BEAGLE-backed [26] `UltrametricTree` for the Felsenstein likelihood – and feeds both implementations from one shared specification file, so a disagreement is a real discrepancy rather than a difference in test setup.

Fourteen cases are specified, chosen to probe the places where two implementations of the same density diverge: multiple alleles per species, GTR with skewed base frequencies and asymmetric exchange rates, stacked reticulations, boundary population sizes, embeddings of probability zero (where both sides must agree on $-\infty$), IUPAC ambiguity codes and gaps, branch-length saturation, fully constant alignments, and a five-taxon caterpillar. The two quantities compared are $\log P(g \mid \Psi)$ and $\log P(S \mid g)$.

All cases agree to within 10^{-5} , and the $-\infty$ case matches exactly. The suite runs as 23 pytest cases and skips cleanly when no PhyloNet jar or BEAGLE installation is present, so it does not burden contributors who have neither.

This is a statement about two likelihood functions, not about two searches. Agreement on a fixed network says nothing about whether the two searches visit the same topologies, and Section 5.3 shows that on reticulate data they do not always find the same answer.

5.2 Other correctness evidence

Table 2 gives the test suite by area. Three of its entries are doing more than counting.

The 432 substitution-model tests exist because an audit of `GTR.py` found that its rate matrix was not time reversible for non-uniform base frequencies, that e^{Qt} was being computed with a symmetric eigensolver on a non-symmetric matrix, and that three of the eight models could not be constructed at all. Every model is now checked against `scipy.linalg.expm` [7]; the largest absolute disagreement across the eight is 4.4×10^{-16} (Table 4). Reporting this is uncomfortable but informative, and the reason no published result changes is worth stating plainly: nothing on the inference path used those classes. The sequence likelihood has its own, separately correct implementation, so the audit cross-validates two implementations that now agree exactly rather than correcting any analysis.

Area	Tests
Substitution models (<code>GTR.py</code>)	432
Network comparison and decomposition	153
Network representation and topology moves	163
Inference API and dispatch registry	88
PhyloNet numerical cross-check	23
Memory and mutation-cost bounds	9
Total collected	1130
Passing in the default run	1125
Deselected as <code>slow</code>	5
Failing	0
Wall time, default run: 46 s	
CI Python versions: 3.9, 3.10, 3.11, 3.12, 3.13, 3.14	
CI jobs: tests, windows, package, slow	

Table 2: Test counts by area, and the continuous-integration configuration. Counts are collected by `benchmarks/bench_testsuite.py`, which runs the suite and parses the result rather than restating it. The skipped count is environment dependent: the PhyloNet cross-check runs only where a jar and BEAGLE are installed, and it did run for the figures reported here.

The incremental pseudo-likelihood scorer – which reuses a cached dynamic program when only branch lengths or inheritance probabilities change – is checked against a full rebuild rather than against expectation, which is what makes the optimisation safe to leave enabled.

The nine tests on memory and mutation cost are regression bounds rather than measurements: they pair timing and allocation limits with structural invariant checks on incidence bookkeeping, so a change that degrades edge-mutation cost or corrupts the indices fails the suite instead of being noticed later.

5.3 Head-to-head against PhyloNet

Gene trees were simulated under the multispecies network coalescent from a known species tree or network, written once, and given to both implementations. Both were asked for the same method – pseudo-likelihood network inference, PhyloNet’s `InferNetwork_MPL` – and both were given a reticulation budget equal to the true count. The population parameter was set to $\theta = 0.5$, which leaves between 46% and none of the gene trees concordant with the generating topology depending on the condition; at the simulator’s smaller default scale every simulated gene tree comes out identical to the species tree, and a comparison on such data measures nothing. Both searches are stochastic, so each of the ten conditions was run on three independently simulated replicates, giving 30 trials per implementation. Table 3 and Figure 2(a) report the aggregate; per-replicate values are in the benchmark JSON.

On runtime, `PHYNETPY` is faster on eight of the ten conditions, by up to about $6\times$ at eight taxa with 200 gene trees, and the margin grows with problem size. The two exceptions are the single-reticulation conditions at five and six taxa, where `PhyNetPy`’s mean runtime is the higher of the two. Because PhyloNet’s figure is self-reported and excludes JVM start-up, the comparison is conservative in PhyloNet’s favour. Both tools were run single-threaded.

The accuracy finding runs the other way, and it is the more consequential of the two. PhyloNet recovered the generating topology exactly in 16 of 30 trials. `PHYNETPY` did so in 8 of 30 under its default settings and 7 of 30 under the `phylonet` preset – roughly half as often. The preset, which

Taxa	Genes	r	Conc.	Mean runtime (s)			Replicates recovering truth		
				PNPY (default)	PNPY (phylonet)	PhyloNet	PNPY (default)	PNPY (phylonet)	PhyloNet
4	25	0	40%	0.11	0.08	0.11	2/3	2/3	2/3
4	100	0	46%	0.09	0.06	0.15	1/3	1/3	2/3
5	50	0	34%	0.12	0.12	0.45	1/3	1/3	2/3
5	200	0	38%	0.12	0.12	0.41	1/3	1/3	2/3
6	100	0	22%	0.19	0.15	0.70	0/3	0/3	2/3
6	400	0	26%	0.17	0.12	0.73	0/3	0/3	1/3
8	200	0	15%	0.41	0.46	2.38	1/3	1/3	3/3
5	100	1	0%	1.11	5.48	0.86	2/3	1/3	1/3
6	200	1	16%	1.08	0.93	1.48	0/3	0/3	0/3
8	200	1	0%	1.23	3.36	4.32	0/3	0/3	1/3
Total							8/30	7/30	16/30

Table 3: PHYNETPY and PhyloNet 3.8.2 running pseudo-likelihood network inference on byte-identical gene trees, each given a reticulation budget equal to the number of reticulations in the generating network. Three independently simulated replicates per condition; the right-hand block counts replicates in which the inferred network equalled the generating network (μ -distance zero). “Conc.” is the mean percentage of simulated gene trees whose topology matches the generating network, so a low value is a harder problem. PHYNETPY is reported under its default search settings and under the **phylonet** preset, which reproduces PhyloNet’s per-topology parameter optimisation. PhyloNet’s runtime is self-reported and excludes JVM start-up.

exists precisely to match PhyloNet’s per-topology parameter optimisation, does not close the gap, and on one condition it costs several times the runtime for no improvement. On this benchmark PHYNETPY’s pseudo-likelihood search is faster and less accurate than PhyloNet’s, and neither tool recovers the truth reliably on the reticulate conditions.

We have not isolated the cause, and it would be premature to attribute it to the search strategy. One observation argues against the simplest explanation, that the search is merely under-run: on an eight-taxon condition it reports a log pseudo-likelihood about two orders of magnitude worse than the score of the generating network itself (-9.2×10^5 against -8.3×10^3), and that value does not change when the iteration budget is raised from its default to 50,000. More iterations would help a search that was stopping early. Two candidate explanations remain open. Continuous parameters on the returned network may be left un-optimised, so a near-correct topology is reported with a poor score. And the pseudo-likelihood criterion does not enforce the branch-length unit discipline that the full likelihood does, so a network carrying lengths on the substitution scale is scored as though they were coalescent units. Both are recorded as defects rather than explained away.²

Two limits on the comparison should be read with it. The simulated data come from PHYNETPY’s own simulator under a model both implementations assume, which makes the comparison reproducible from this repository and makes the runtime measurement fair, but says nothing about robustness to model misspecification. And while three replicates are enough to show that the accuracy difference is not an artefact of a single lucky run, they are not enough to estimate its size precisely; the counts in Table 3 should be read as indicating a gap of roughly a factor of two rather than as a calibrated effect size.

Model	Implementation	PHYNETPY (μ s)	expm (μ s)	Ratio	max $ \Delta $
JC	closed form	1.68	24.75	14.7 $\times$	1.1e-16
F81	closed form	1.32	24.96	18.9 $\times$	1.4e-17
K81	closed form	1.45	25.95	18.0 $\times$	1.1e-16
K80	Tamura-Nei	2.03	24.70	12.1 $\times$	1.1e-16
HKY	Tamura-Nei	2.06	23.74	11.5 $\times$	2.8e-17
TN93	Tamura-Nei	2.04	24.78	12.1 $\times$	1.1e-16
SYM	cached eigendecomposition	2.67	23.91	8.9 $\times$	2.1e-16
GTR	cached eigendecomposition	2.60	25.45	9.8 $\times$	4.4e-16

Table 4: Transition-matrix evaluation at $t = 0.1$ against a fresh `scipy.linalg.expm` call on the same rate matrix. Six models have a closed form; SYM and GTR cache an eigendecomposition. The final column is the largest absolute elementwise disagreement with `expm`, so this table is simultaneously a speed and a correctness measurement.

Operation	PHYNETPY (μ s)	NetworkX (μ s)	Ratio (NetworkX / PHYNETPY)
Incident in-edges of a node	0.16	1.81	11.45 $\times$
Out-degree of a node	0.16	0.20	1.21 $\times$
Enumerate all leaves	173.41	453.99	2.62 $\times$
Topological order	2384.32	1295.26	0.54 $\times$
Leaf descendants, all nodes	1452.37	—	—
Peak construction memory (KiB)	2103	1111	0.53 $\times$

Table 5: The compiled graph core against `networkx.DiGraph` [27] on the same 1,999-node topology. Ratios above one favour PHYNETPY. PHYNETPY is faster on the incidence and leaf queries that dominate network search and slower on topological ordering, and it uses roughly twice the peak memory during construction.

5.4 Component measurements

Three component measurements explain where PHYNETPY’s time goes.

Transition-matrix evaluation (Table 4) is between 8.5 $\times$ and 19 $\times$ faster than calling `expm`, because six of the eight models use a closed form and the remaining two cache an eigendecomposition. This is not a claim to have improved on SciPy; it is the expected benefit of exploiting structure that a general matrix exponential cannot see.

The compiled graph core (Table 5) is a genuinely mixed result. Incidence lookup is about 11 $\times$ faster than NetworkX and leaf enumeration about 3.8 $\times$, both essentially independent of network size, which is the payoff from maintaining incidence indices. Topological ordering is about 0.7 $\times$ – that is, slower. And peak memory during construction is roughly 1.9 $\times$ NetworkX’s. The operations PHYNETPY optimises are the ones a search loop performs most often, and the trade it makes is memory for lookup speed; a reader whose workload is dominated by whole-graph traversals rather than incidence queries should not expect a gain.

The compiled pseudo-likelihood kernel (Table 6) is worth between 1.3 $\times$ and 1.7 $\times$ over the pure-Python dynamic program, with the advantage growing with taxon count, and the two paths agree

²[TODO: resolve before submission. Either fix the underlying cause and re-run this benchmark, or keep this subsection as written and add the diagnosis. Do not soften the accuracy result without new measurements.]

Taxa	Genes	Compiled (ms)	Python (ms)	Ratio	$ \Delta \log \text{PL} $
6	40	1.13	1.46	$1.29\times$	1.1e-13
8	60	2.97	4.69	$1.58\times$	1.1e-12
10	80	7.10	11.08	$1.56\times$	9.1e-13
12	100	12.93	21.70	$1.68\times$	2.9e-11

Table 6: The compiled pseudo-likelihood dynamic program against the pure-Python reference that remains in the repository, selected by a module flag so that only the inner loop differs. The final column confirms the two paths return the same score.

Taxa	Sites	Time per score (ms)	$\log L$
3	100	0.37	-106.2
3	1000	1.20	-1030.6
3	5000	4.94	-5161.8
4	100	0.69	-114.2
4	1000	1.73	-1067.0
4	5000	7.94	-5774.7
5	100	0.70	-120.9
5	1000	2.40	-1196.6
5	5000	11.89	-6461.3

Table 7: Exact biallelic-marker likelihood of a fixed network, median of repeated evaluations with the likelihood model already constructed. The single-evaluation cost, which includes that construction, is recorded alongside these figures in `benchmarks/snp.json`.

to within 2×10^{-12} . That is a smaller factor than compiled-versus-interpreted comparisons often report, because most of the work is already in array operations rather than Python-level loops.

Biallelic-marker likelihood evaluation (Table 7, Figure 2(b)) takes about 1.1 ms for three taxa and 1,000 sites, scaling roughly linearly in sites and more steeply in taxa, as the exact algorithm’s dependence on the number of sampled lineages implies. The first evaluation in a process costs noticeably more than subsequent ones because the likelihood model is constructed on first use; both figures are recorded, since a single interactive score and a score inside a search loop are different quantities.

5.5 What is not measured

Three gaps should be read alongside the tables above. No comparison against PhyloNet’s Bayesian samplers is reported: matching two MCMC implementations requires agreeing on chain length, proposal weights and convergence criteria before any timing is meaningful, and that comparison has not been done. No empirical dataset is analysed here. And every measurement comes from one machine, so the ratios should be read as indicative rather than as portable constants.

6 Moving from PhyloNet to PhyNetPy

This section is addressed to researchers with working PhyloNet analyses. The useful question is not which tool is better but what specifically changes, and what does not carry over.

6.1 What changes

The driver file disappears. A PhyloNet analysis is a NEXUS file with a `PHYLONET` block, executed through `java -jar`. Parameters are positional and flag-based inside that block, and a sweep over conditions means generating files. In `PHYNETPY` the analysis is Python: a loop over θ values or reticulation budgets is a loop, and intermediate results stay as objects.

Choosing a method becomes choosing arguments. Switching from maximum likelihood to pseudo-likelihood changes one argument rather than one command name, and the return type does not change with it. In PhyloNet the corresponding change is to a different command with its own syntax and output format.

Results are objects, not text to parse. PhyloNet writes inferred networks and scores to standard output, and a pipeline around it is usually a parser. `infer` returns an `InferenceResult` whose `.best` is a `Network` that the distance functions, the decomposition routines and `score` all accept directly. This is the practical difference most PhyloNet users will notice first.

Unsupported requests say which kind of unsupported they are. The three-way distinction between `TypeError`, `ValueError` and `NotImplementedError` tells a user whether to change the data, change the criterion, or wait.

6.2 What does not carry over

Input files are not automatically compatible. `PHYNETPY` reads extended Newick, NEXUS, FASTA and VCF, so the *data* in a PhyloNet NEXUS file is usually readable. The `PHYLONET` command block is not: there is no translator from a PhyloNet block to a `PHYNETPY` call, and the migration is a rewrite of the driver, not a conversion.

Results will not be identical. The two implementations agree on the likelihood of a fixed network to 10^{-5} (Section 5), but searches are stochastic and their proposal kernels differ, so inferred networks can differ — and in Section 5.3 they did on one reticulate condition. The `phylonet` search preset exists to narrow this gap when the point is a comparison rather than an analysis, by reproducing PhyloNet’s optimise-every-parameter-per-topology behaviour.

Some analyses are not available. Maximum parsimony from gene trees under diploid hybridization is absent, and marker maximum likelihood can score but not search. Anyone whose work depends on those should continue to use PhyloNet.

Installation requirements differ. PhyloNet needs a JVM and a jar. `PHYNETPY` installs with `pip`, but the compiled graph core is required, so installing from a source distribution needs a C compiler. Where a wheel is published for the target platform this does not arise.

6.3 A worked correspondence

A PhyloNet pseudo-likelihood analysis reads, inside a NEXUS file:

```
BEGIN PHYLONET;
InferNetwork_MPL (gt1,gt2,...,gt100) 1 -pl 4;
END;
```

The corresponding PHYNETPY analysis:

```
from phynetpy.data import GeneTrees
from phynetpy.criteria import PseudoLikelihood
from phynetpy.infer import infer

gts = GeneTrees.from_file("genes.nex", {"A": ["A1", "A2"], "B": ["B1"]})
result = infer(gts, criterion=PseudoLikelihood(), max_reticulations=1)
```

The species-to-allele mapping moves from a `-a` flag into the data object, where it travels with the data it describes instead of being restated per command. The reticulation budget is a keyword rather than a positional integer. The gene trees are named once, by file, rather than enumerated in the call.

These two are not equivalent in every respect, and the differences are the point of this section rather than a caveat to it: the searches differ, the defaults differ, and `-pl 4` requests four-way parallelism from PhyloNet whereas the PHYNETPY call above is single-threaded. A migration should re-establish results on data with a known answer rather than assume the two calls are interchangeable.

7 Example workflows

Every listing below is an excerpt of a function in `paper/phynetpy-1.0.0/listings/paper_listings.py`, which runs as a script and fails loudly if any listing stops matching the API. Figure 4 shows how the pieces fit together.

7.1 Building and inspecting a network

Reticulations are written in extended Newick and the resulting object supports the structural queries directly.

```
from phynetpy import blobs, count_reticulations, level, read_newick

net = read_newick("((A:0.1,(B:0.2)#H1:0.1):0.3,(#H1:0.2,C:0.3):0.2);")

count_reticulations(net)    # 1
level(net)                  # 1
net.is_acyclic()            # True
len(blobs(net))             # 4
```

The same type can be assembled node by node, which is what a method that proposes topologies does internally:

```
from phynetpy import Edge, Network, Node

root, internal = Node("Root"), Node("I1")
a, b, c = Node("A"), Node("B"), Node("C")
net = Network()
net.add_nodes([root, internal, a, b, c])
for edge in (Edge(root, internal), Edge(root, c),
             Edge(internal, a), Edge(internal, b)):
    net.add_edges(edge)
```

7.2 Scoring a network, then inferring one

This listing shows the unit discipline in use. The simulator writes branch lengths in expected substitutions per site; the gene-tree criteria read coalescent units. Converting is a call, not an assumption.

```
from phynetpy import BranchLengthUnit, convert_network_branch_lengths
from phynetpy.criteria import Likelihood, PseudoLikelihood
from phynetpy.infer import infer, score, simulate
from phynetpy.models import MSC

theta = 0.5
gts = simulate(MSC(theta=theta), taxa=6, n=200, data="gene_trees", seed=3)
truth = convert_network_branch_lengths(
    gts.true_network, BranchLengthUnit.COALESCENT_2N, theta=theta
)

score(truth, gts, model=MSC(theta=theta), criterion=Likelihood())
# -514.956
score(truth, gts, model=MSC(theta=theta), criterion=PseudoLikelihood())
# -1883.194

result = infer(gts, criterion=PseudoLikelihood(), max_reticulations=0)
result.method      # 'InferNetwork_MPL'
result.score       # -1882.494
```

Changing criterion to Bayesian() turns the same call into a posterior sample, and result.posterior is then populated. The rest of the call, and the type it returns, do not change.

7.3 Simulation and recovery

Because simulate produces a data-axis object, a recovery check is two lines. This is the pattern behind the benchmark in Section 5.3 and behind Examples/sim_recovery.py.

```
from phynetpy.GraphUtils import mu_distance
from phynetpy.infer import infer, simulate

sim = simulate(MSC(theta=0.5), taxa=6, n=200, data="gene_trees", seed=11)
recovered = infer(sim, criterion=PseudoLikelihood(), max_reticulations=0)

mu_distance(sim.true_network, recovered.best) # 0
```

The generating network is attached to the simulated data as .true_network, so the comparison does not require tracking it separately.

7.4 Inspecting and extending the method set

The set of available methods is queryable rather than documented separately:

```
from phynetpy.infer import validity_matrix

for data, row in validity_matrix().items():
    print(data, row)
```

```

GeneTrees      {'MDC': '-', 'Likelihood': 'InferNetwork_ML',
                'PseudoLikelihood': 'InferNetwork_MPL', 'Bayesian': 'MCMC_GT'}
Alignment      {'MDC': 'x', 'Likelihood': '-',
                'PseudoLikelihood': 'x', 'Bayesian': 'MCMC_SEQ'}
BiallelicMarkers {'MDC': 'x', 'Likelihood': 'MLE_BiMarkers',
                  'PseudoLikelihood': '-', 'Bayesian': 'MCMC_BiMarkers'}

```

Adding a criterion means declaring what it accepts and registering an engine for it. The objective below is deliberately trivial – total branch length – so that the mechanism rather than the statistics is visible:

```

from phynetpy.criteria import Criterion
from phynetpy.data import GeneTrees
from phynetpy.infer import Engine, register, score
from phynetpy.models import MSC

class TreeLength(Criterion):
    accepts_data = (GeneTrees,)
    scorable = True
    use_branch_lengths = False

@register(GeneTrees, MSC, TreeLength)
class TreeLengthEngine(Engine):
    method = "TreeLength"

    def infer(self, data):
        raise NotImplementedError("scoring only")

    def score(self, network, data, optimize=False):
        return sum(e.get_length() or 0.0 for e in network.E())

```

After that, `score(net, data, criterion=TreeLength())` dispatches to the new engine, `validity_matrix()` shows the new column, and an unregistered combination involving `TreeLength` raises `NotImplementedError` rather than silently falling back. A real criterion would additionally implement `infer`, and would get the search drivers, topology moves and parameter optimiser from the shared layer rather than reimplementing them.

8 Discussion

8.1 Infrastructure as the contribution

The methods in Table 1 are mostly not new. What is new is that they sit on shared components: one network type, one set of topology moves, one parameter optimiser, one I/O layer, one set of chain diagnostics. A developer adding a criterion writes the criterion.

Whether this lowers the barrier enough to change how network methods get built is not something this paper can settle. The registry is exercised by seven methods, all of them ours, and an interface that suffices for its author is weaker evidence than one an outside contributor found sufficient. The design argument is stated in Section 3; the measurement that would test it does not exist yet.

Two design decisions seem worth keeping regardless. Making the validity of a (data, model, criterion) triple a runtime query, rather than prose in documentation, means the method set cannot drift from its description – Table 1 is generated from the registry. And separating the three failure

modes gives a user an actionable answer: change the data, wait for a release, or reconsider the question.

8.2 What the Python implementation does and does not buy

Being a Python library makes some things straightforward that were awkward before. An inferred network is an object that goes directly into a distance function, a plotting call, or a pandas frame. A sweep over reticulation counts is a loop. Simulation and inference compose without an intermediate file, which is what makes the recovery check in Section 7 two lines.

It does not, by itself, make anything faster. PHYNETPY’s runtime advantage in Section 5.3 comes from compiled kernels and from algorithmic choices such as incremental rescaling, not from the choice of language – and the component measurements show the trade is not uniform: the graph core is faster on incidence queries and slower on whole-graph traversal, using about twice the memory. Claims that one language is faster than another are not what this section is for.

Integration with the numerical and machine-learning ecosystem is often cited as a reason to work in Python, and the architecture does make it practical: the network is an ordinary object, likelihoods are ordinary callables, and a scorer backed by PyTorch or JAX could be registered as a cell without changing anything else. PHYNETPY 1.0.0 contains no machine-learning functionality, no differentiable likelihood and no GPU support beyond an optional CuPy path in the marker likelihood. The integration is available rather than done.

8.3 Reproducibility

Two properties of the release matter for reproducing results. The version is single-sourced and CI verifies that a clean install of both the wheel and the source distribution reports it, so a paper citing a version is citing something checkable. And the parts of an analysis that affect results – the model parameters, the criterion, the search preset, the seed – are arguments at the call site rather than defaults buried in a driver file.

The benchmark scripts for this paper follow the same discipline: each writes a JSON file recording package versions, interpreter, platform, replicate counts and seeds, and the tables and figures are generated from those files. This is why the accuracy result in Section 5.3 is reported as it is. It emerged from re-running a benchmark rather than from reading code, and a pipeline that only regenerated favourable numbers would not have surfaced it.

8.4 Limitations

Relative to PhyloNet, PHYNETPY 1.0.0 lacks maximum parsimony from gene trees under diploid hybridization, and its marker maximum likelihood can score a network but not search. Both are visible in `validity_matrix()`.

The accuracy deficit in Section 5.3 is the most consequential open item. Across 30 trials of matched pseudo-likelihood inference PhyloNet recovered the generating topology 16 times and PHYNETPY 8, and the `phylonet` preset intended to make the comparison like-for-like did not close the gap. Anyone whose work depends on pseudo-likelihood network inference should validate against PhyloNet on data with a known answer before relying on PHYNETPY for it.

Other limits are narrower. Across-site rate heterogeneity is not implemented, so sequence analyses cannot fit a discretised gamma over sites. Simulation supports only the MSC model; the allopolyploid model has no simulator. The network simulator grafts reticulations onto a separately drawn tree rather than running a birth–death–hybridization process, so it should not be used to study the distribution of network shapes. Visualisation is ASCII text. There is no network classifier

for the graph-theoretic classes beyond level, and no summarisation of *sets* of networks, which is what a Bayesian run most wants. One compiled kernel, `seq_engine_cy`, is built but not yet called. Installing from source needs a C compiler. And every measurement in this paper comes from one machine.

8.5 Future work

The immediate items are the two unbuilt cells and a diagnosis of the accuracy result. Beyond those: connecting the compiled sequence kernel; rate heterogeneity; a birth–death–hybridization simulator; network classification and set-level summarisation; and visualisation, where writing Tracer and NEXUS output has so far been a substitute for having any of our own.

The architecture suggests directions we have not taken. A differentiable pseudo-likelihood would make gradient-based optimisation over continuous parameters available; a learned proposal distribution would fit the `ProposalKernel` interface; approximate Bayesian computation would compose with `simulate` directly. These are plausible because the pieces are separable, not because any of them is underway.

9 Conclusions

PHYNETPY 1.0.0 provides a Python foundation for phylogenetic network work: a reusable network type, an analysis layer that does not depend on any inference method, and seven inference methods reached through two verbs over three typed axes. Six of those methods correspond to PhyloNet analyses, and the likelihood functions underlying two of them agree with PhyloNet’s own to 10^{-5} across fourteen adversarial cases.

It is a successor to PhyloNet in the sense that it continues the same line of software and reimplements much of the same body of methods. It is not a replacement. Two PhyloNet analyses have no counterpart here, PhyloNet command files do not carry over, and on the one search we compared directly PhyloNet recovered the true topology about twice as often as we did. Where those matter, PhyloNet remains the right tool.

The case for building on PHYNETPY is about what the next method costs. A new criterion is a class and a registry entry, and it inherits the network representation, the topology moves, the parameter optimiser and the diagnostics rather than restating them.

10 Reproducibility and software citation

10.1 Reproducing the results in this paper

Every number, table and figure in Sections 5 and 7 is produced by scripts in `paper/phynetpy-1.0.0/`. Running

```
export PHYLONET_JAR=/path/to/phylonet.jar
python paper/phynetpy-1.0.0/benchmarks/run_all.py
python paper/phynetpy-1.0.0/benchmarks/make_analyses_table.py
python paper/phynetpy-1.0.0/listings/paper_listings.py
```

regenerates the benchmark JSON, rewrites `tables/*.tex` and `figures/performance.pdf` from it, rebuilds Table 1 from the live dispatch registry, and executes every code listing. The PhyloNet comparison is skipped with a warning if no jar is configured; the rest needs nothing beyond the

package and its dependencies. The tables under `tables/` are generated files and carry a header saying so.

10.2 Environment

Measurements in this paper were taken under the following configuration, which each benchmark script also records in the `meta` block of its own JSON output:

- PHYNETPY 0.6.0.³
- CPython 3.14.2 on Windows 11 (build 10.0.26200), AMD64.
- AMD Family 25 Model 97 (Zen 4 class), single-threaded unless stated.
- NumPy 2.3.5, SciPy 1.16.3, NetworkX 3.6.1.
- PhyloNet 3.8.2 (`phylonet.jar`, 41,019,851 bytes) under Java 1.8.0_491, with BEAGLE for the Felsenstein cross-check.

Random seeds are fixed in every script and recorded per measurement. The head-to-head comparison runs three replicates per condition, replicate i using seed $20260925 + i$ for both simulation and PHYNETPY’s search; the component benchmarks use seeds 7 and 11, and the marker timings use seed 1. PhyloNet’s search is stochastic and its command-line interface is not seeded here, so its variation across replicates is uncontrolled – which is part of why the comparison is replicated rather than run once. The generating network for each replicate is stored as Newick in the benchmark JSON alongside its results, together with the gene-tree concordance that replicate produced, so any trial can be reconstructed exactly.

Timing protocol. Microbenchmarks report the median of repeated batches measured with `time.perf_counter`, after one warm-up call; loop and repeat counts are stored per measurement. Interpreter start-up and module import are excluded and recorded separately. For the PhyloNet comparison, PhyloNet’s self-reported running time is used because it excludes JVM start-up and is therefore comparable to PHYNETPY’s in-process timing; total wall time including JVM start-up is also recorded in the JSON. No result in this paper uses multi-threading or GPU offload.

Datasets. All data are simulated by PHYNETPY itself, from seeds recorded in the scripts, so no external archive is required. No empirical dataset is analysed in this paper.

10.3 Citing PhyNetPy

- Software: PHYNETPY version 1.0.0.
- Repository: <https://github.com/NakhlehLab/PhyNetPy>, tag `v1.0.0`.⁴
- Package: <https://pypi.org/project/phynetpy/>.
- License: MIT.

³[TODO: re-run every benchmark against the tagged 1.0.0 release and update this section, the abstract and Section 5. The measurements reported here were taken on the 0.6.0 codebase, which is 1.0.0 without the two unbuilt cells of Table 1.]

⁴[TODO: create the `v1.0.0` tag and confirm this reference before submission; the current release tag is `v0.6.0`.]

- Archived release: [28].

Methods reimplemented from published work should be cited to their original sources rather than to PHYNETPY or to PhyloNet generically; the relevant citation for each is given in Table 1.

Acknowledgements

This work was supported by the National Science Foundation under award IIBR 2420499.

References

- [1] TODO. TODO — a review of phylogenetic networks and reticulate evolution, for the introduction. huson & scornavacca or a degnan-style review. *TODO*, 0000.
- [2] TODO. TODO — a review of incomplete lineage sorting and gene-tree discordance, for the background section. *TODO*, 0000.
- [3] TODO. TODO — the primary reference for the multispecies network coalescent. phynetpy’s msc model axis implements this; cite the work that introduced the coalescent-with-hybridization density, not phylonet. *TODO*, 0000.
- [4] Cuong Than, Derek Ruths, and Luay Nakhleh. PhyloNet: a software package for analyzing and reconstructing reticulate evolutionary relationships. *BMC Bioinformatics*, 9:322, 2008. doi: 10.1186/1471-2105-9-322.
- [5] Dingqiao Wen, Yun Yu, Jiafan Zhu, and Luay Nakhleh. Inferring phylogenetic networks using PhyloNet. *Systematic Biology*, 67(4):735–740, 2018. doi: 10.1093/sysbio/syy015.
- [6] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.
- [7] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [8] Jeet Sukumaran and Mark T. Holder. DendroPy: a Python library for phylogenetic computing. *Bioinformatics*, 26(12):1569–1571, 2010. doi: 10.1093/bioinformatics/btq228.
- [9] Jaime Huerta-Cepas, François Serra, and Peer Bork. ETE 3: reconstruction, analysis, and visualization of phylogenomic data. *Molecular Biology and Evolution*, 33(6):1635–1638, 2016. doi: 10.1093/molbev/msw046.
- [10] Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009. doi: 10.1093/bioinformatics/btp163.
- [11] Cuong Than and Luay Nakhleh. Species tree inference by minimizing deep coalescences. *PLoS Computational Biology*, 5(9):e1000501, 2009. doi: 10.1371/journal.pcbi.1000501.

- [12] Zhi Yan, Peng Du, Matthew W. Hahn, and Luay Nakhleh. Maximum parsimony inference of phylogenetic networks in the presence of polyploid complexes. *Systematic Biology*, 2022. doi: 10.1093/sysbio/syab081. InferNetwork_MP_Allopp, implemented in PhyloNet. TODO — add volume, issue and page range.
- [13] Yun Yu, Jianrong Dong, Kevin J. Liu, and Luay Nakhleh. Maximum likelihood inference of reticulate evolutionary histories. *Proceedings of the National Academy of Sciences*, 111(46): 16448–16453, 2014. doi: 10.1073/pnas.1407950111. TODO — verify author list, volume and page range against the published article. This is the reference behind the InferNetwork_ML cell.
- [14] Yun Yu and Luay Nakhleh. A maximum pseudo-likelihood approach for phylogenetic networks. *BMC Genomics*, 16(Suppl 10):S10, 2015. doi: 10.1186/1471-2164-16-S10-S10. TODO — verify supplement number, article number and DOI. This is the reference behind the InferNetwork_MPL cell.
- [15] Peter J. Green. Reversible jump Markov chain Monte Carlo computation and Bayesian model determination. *Biometrika*, 82(4):711–732, 1995. doi: 10.1093/biomet/82.4.711.
- [16] W. Keith Hastings. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, 57(1):97–109, 1970. doi: 10.1093/biomet/57.1.97.
- [17] Dingqiao Wen and Luay Nakhleh. Coestimating reticulate phylogenies and gene trees from multilocus sequence data. *Systematic Biology*, 67(3):439–457, 2018. doi: 10.1093/sysbio/syx085. TODO — verify volume, issue and page range. This is the reference behind the MCMC_SEQ cell.
- [18] David Bryant, Remco Bouckaert, Joseph Felsenstein, Noah A. Rosenberg, and Arindam Roy-Choudhury. Inferring species trees directly from biallelic genetic markers: bypassing gene trees in a full coalescent analysis. *Molecular Biology and Evolution*, 29(8):1917–1932, 2012. doi: 10.1093/molbev/mss086.
- [19] Jiafan Zhu, Dingqiao Wen, Yun Yu, Heidi M. Meudt, and Luay Nakhleh. Bayesian inference of phylogenetic networks from bi-allelic genetic markers. *PLoS Computational Biology*, 14(1): e1005932, 2018. doi: 10.1371/journal.pcbi.1005932. TODO — verify volume, issue and article number. Reference behind the MLE_BiMarkers and MCMC_BiMarkers cells.
- [20] Dingqiao Wen, Yun Yu, and Luay Nakhleh. Bayesian inference of reticulate phylogenies under the multispecies network coalescent. *PLoS Genetics*, 12(5):e1006006, 2016. doi: 10.1371/journal.pgen.1006006. TODO — verify. This is the reference behind the MCMC_GT cell; confirm whether MCMC_GT should cite this or Wen & Nakhleh 2018.
- [21] Gabriel Cardona, Mercè Llabrés, Francesc Rosselló, and Gabriel Valiente. Metrics for phylogenetic networks I: generalizations of the Robinson–Foulds metric. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 6(1):46–61, 2009. doi: 10.1109/TCBB.2008.70. TODO — verify. This is the reference for the mu-distance implemented as GraphUtils.mu_distance.
- [22] Berk Yakici, Huw A. Ogilvie, and Luay Nakhleh. TODO — the recomb-cg 2022 paper introducing average path distance (apd) and weighted apd, implemented as graphutils.average_path_distance and weighted_average_path_distance. establish the exact title, proceedings volume and pages. 2022.

- [23] John Geweke. Evaluating the accuracy of sampling-based approaches to the calculation of posterior moments. *Bayesian Statistics*, 4:169–193, 1992. TODO — this is a book chapter in Bernardo et al., *Bayesian Statistics 4* (Oxford University Press). Convert to an @incollection with editors and publisher.
- [24] Andrew Gelman and Donald B. Rubin. Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4):457–472, 1992. doi: 10.1214/ss/1177011136.
- [25] Andrew Rambaut, Alexei J. Drummond, Dong Xie, Guy Baele, and Marc A. Suchard. Posterior summarization in Bayesian phylogenetics using Tracer 1.7. *Systematic Biology*, 67(5):901–904, 2018. doi: 10.1093/sysbio/syy032.
- [26] Daniel L. Ayres, Aaron Darling, Derrick J. Zwickl, et al. BEAGLE: an application programming interface and high-performance computing library for statistical phylogenetics. *Systematic Biology*, 61(1):170–173, 2012. doi: 10.1093/sysbio/syr100.
- [27] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.
- [28] Mark Kessler and Luay Nakhleh. TODO — the archival record for phynetpy 1.0.0. deposit the tagged release with zenodo (or another archive), then replace this entry with the resulting doi. do not invent a doi, 2026.

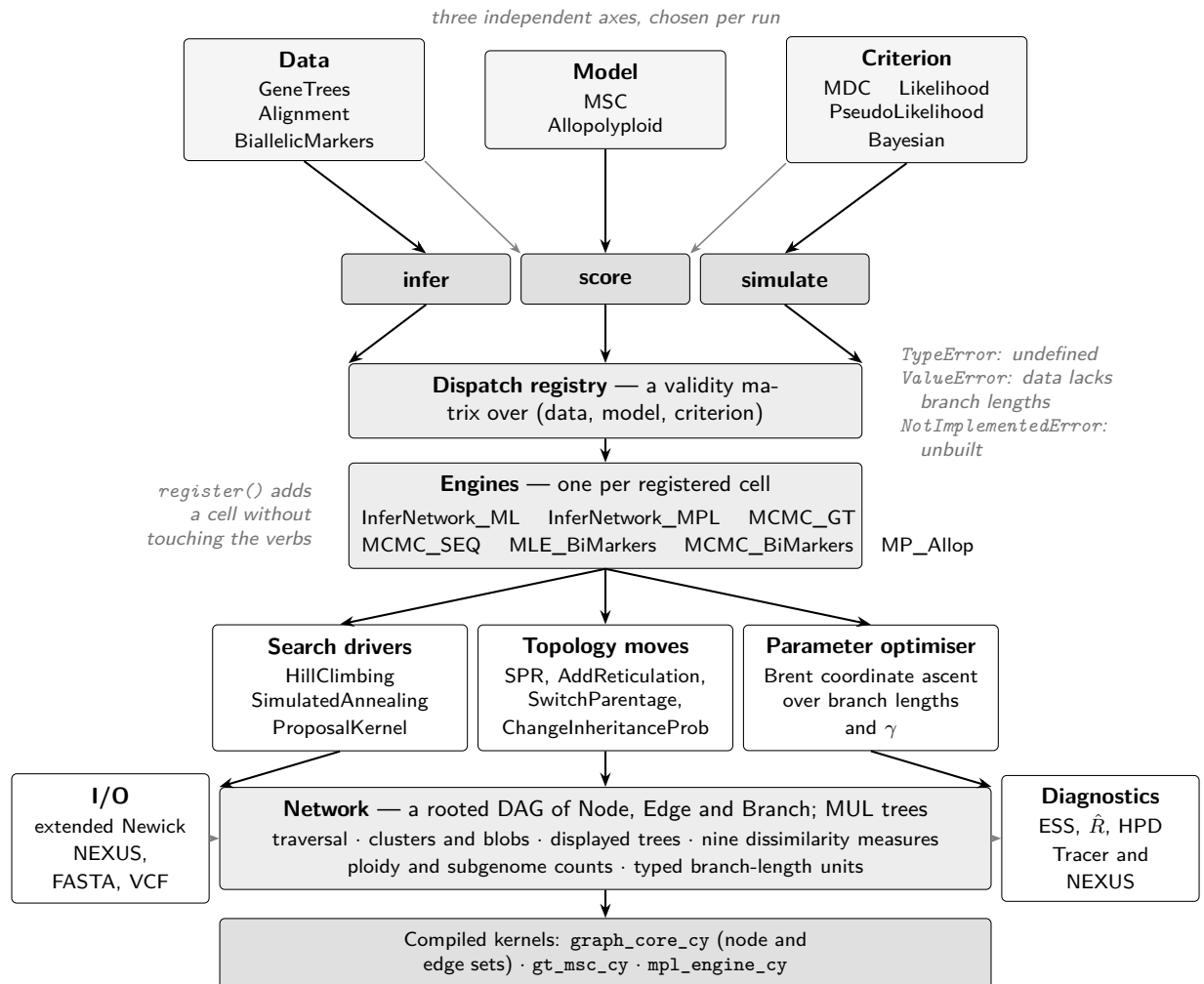


Figure 1: PHYNETPY organised by dependency. A run is specified by picking one object from each of the three axes at the top and calling one of the three verbs. The registry resolves that triple to an engine. Engines do not implement their own search machinery, topology moves, parameter optimisation or network operations; they compose the shared components below them. The compiled kernels sit under the network layer and are not part of the public interface.

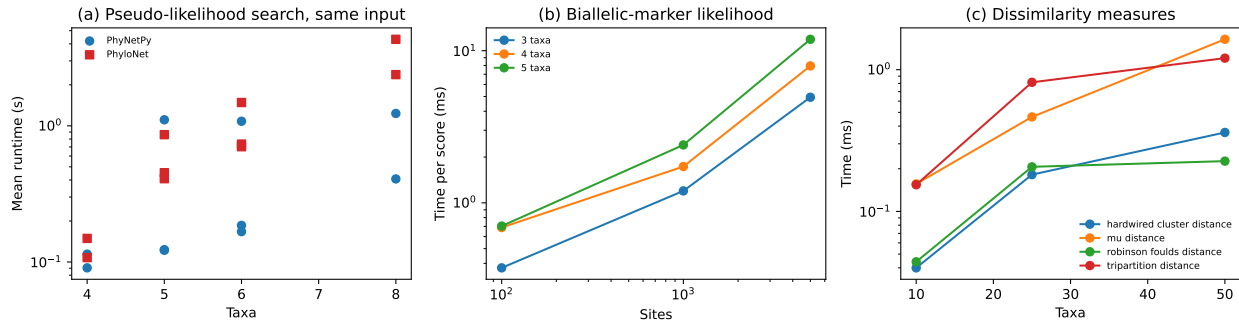


Figure 2: (a) Runtime of pseudo-likelihood inference on identical input, on a log scale, one point per condition of Table 3. Points are not joined because conditions sharing a taxon count differ in the number of gene trees. (b) Time to evaluate the exact biallelic-marker likelihood of a fixed network against the number of sites. (c) Dissimilarity-measure runtime against taxon count. Panels show runtime only; accuracy is in Table 3. Every panel is drawn from the JSON written by the benchmark scripts.

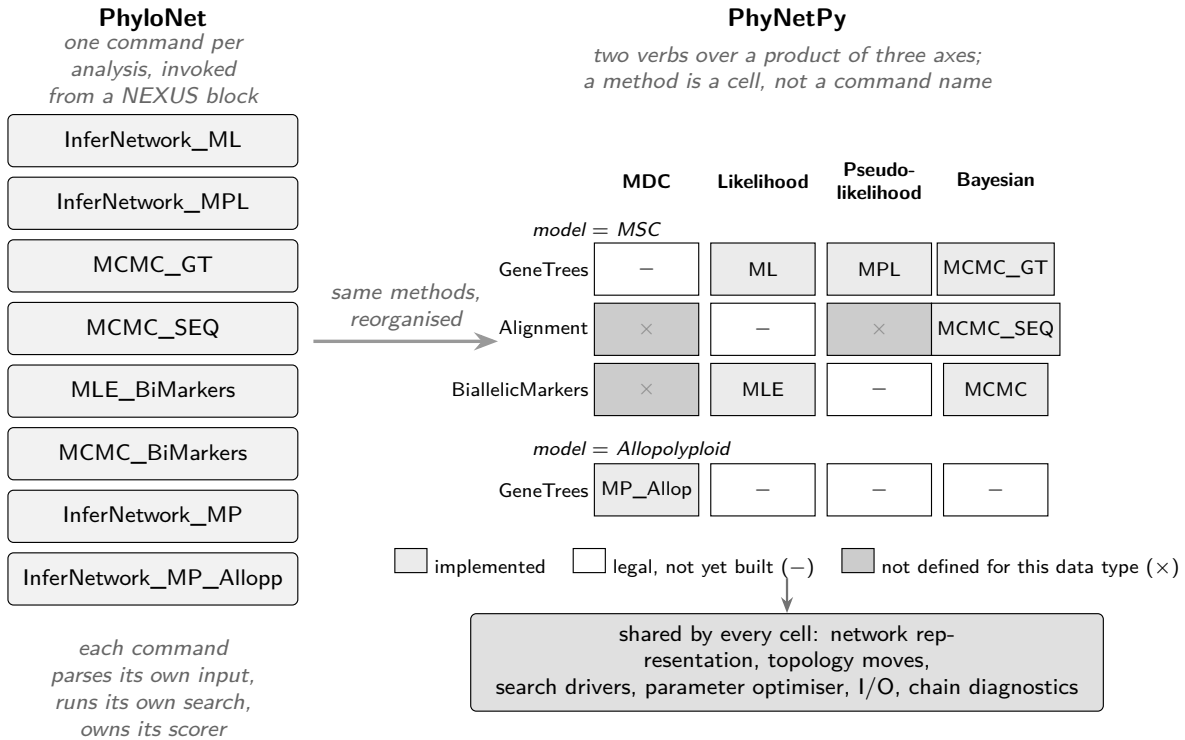


Figure 3: The same body of methods under two organisations. On the left each analysis is a named command that handles its own input, search and scoring. On the right the analyses are cells of a product of three axes over shared machinery, so the empty cells are visible and distinguishable: — marks a combination that is well defined but unbuilt, × one that is not defined for that data type. The figure makes no claim that one organisation produces better inferences.

	PhyloNet 3.8.2	PhyNetPy 1.0.0
Implementation language	Java	Python, with four Cython extension modules
Invocation	A NEXUS file containing a <code>PHYLONET</code> block, run through <code>java -jar</code>	Python function calls; no driver file
Programmatic API	Java classes are reachable by writing Java against the jar	The documented surface is the package itself: 121 exported names, two inference verbs plus <code>simulate</code>
Analysis organisation	One named command per analysis	A method is a (data, model, criterion) cell; dispatch goes through a registry that also reports validity
Adding a method	Implement a new command	Register one cell with <code>@register</code> ; search drivers, moves, parameter optimiser and I/O are inherited
Network representation	Internal to the tool	Network : a reusable rooted DAG, independent of any analysis, with no cap on reticulation in-degree
Distinguishing invalid from unbuilt requests	An unsupported combination surfaces as a command-level error	<code>TypeError</code> , <code>ValueError</code> and <code>NotImplementedError</code> are raised for three distinct causes
<i>Analyses</i>		
Maximum likelihood from gene trees	<code>InferNetwork_ML</code>	Available
Maximum pseudo-likelihood from gene trees	<code>InferNetwork_MPL</code>	Available
Bayesian from gene trees	<code>MCMC_GT</code>	Available
Bayesian from sequences	<code>MCMC_SEQ</code>	Available
Marker likelihood	<code>MLE_BiMarkers</code>	Scoring available; the search path raises
Bayesian from markers	<code>MCMC_BiMarkers</code>	Available
Maximum parsimony, diploid hybridization	<code>InferNetwork_MP</code>	Not in 0.6.0; scheduled for 1.0.0
Maximum parsimony under polyploidy	<code>InferNetwork_MP_Allopp</code>	Available, with bootstrapping
<i>Supporting capability</i>		
Simulation	Available as separate commands	<code>simulate</code> inverts the same axes and returns a data object that feeds <code>infer</code> directly; gene trees, sequences and markers under <code>MSC</code> only
Network comparison	Distance commands are provided	Nine dissimilarity measures plus a reticulation-specific comparison module (tripartitions, precision and recall)
Chain diagnostics	Output is written for external tools	ESS, autocorrelation time, HPD intervals, Geweke and Gelman–Rubin in-process; Tracer and NEXUS output for external tools
Visualisation	[TODO: verify what PhyloNet 3.8.2 provides]	ASCII rendering only
Across-site rate heterogeneity	[TODO: verify]	Not implemented
<i>Distribution and testing</i>		
Installation	Download a jar; requires a JVM	<code>pip install phynetpy</code> ; requires a C compiler when building from source
Automated testing	[TODO: verify what PhyloNet publishes]	1,130 tests, 1,125 passing in the default run
Continuous integration	[TODO: verify]	GitHub Actions across Python 3.9–3.14, plus Windows and packaging jobs
Numerical cross-validation	—	14 cases checked against PhyloNet itself to a tolerance of 10^{-5}
License	[TODO: verify PhyloNet’s license]	MIT

Table 8: PhyloNet 3.8.2 and PHYNETPY 1.0.0. PHYNETPY entries are verified against the source; entries marked [**TODO**] could not be established from the material available and must be checked against PhyloNet’s documentation before submission rather than guessed.

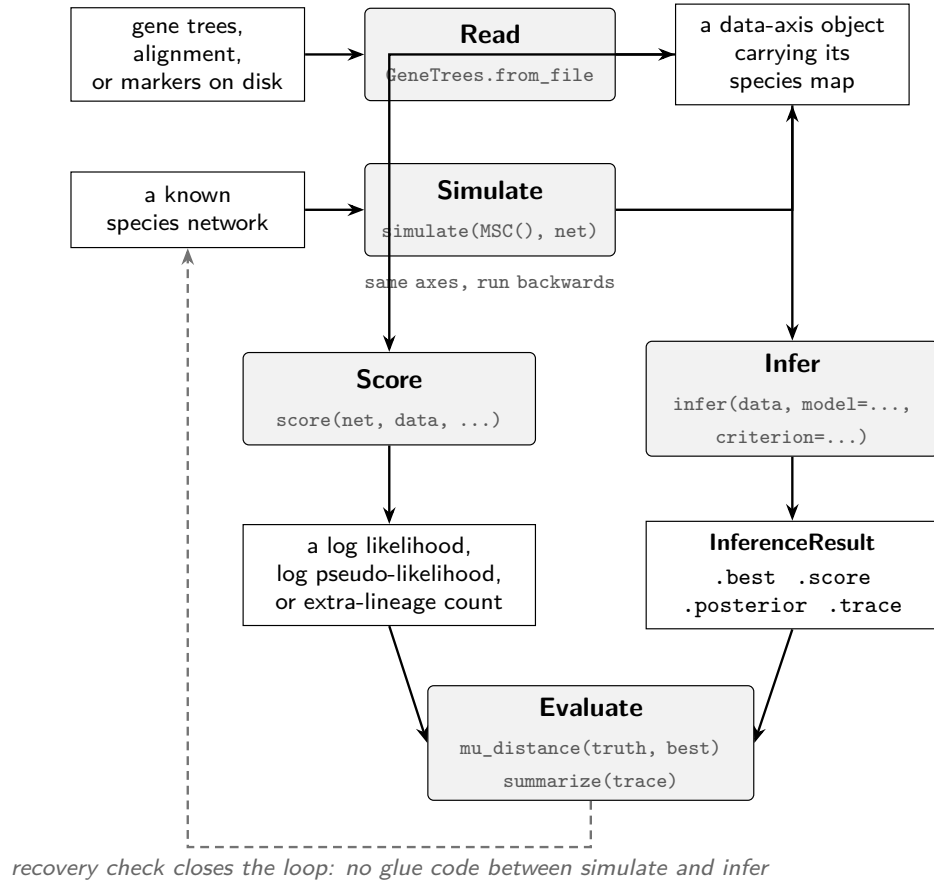


Figure 4: A simulate–infer–evaluate loop in the public API. Data enters either by reading a file or by simulating from a known network; the two verbs act on it; the evaluation step closes the loop. Because `simulate` returns the same kind of object that `infer` consumes, a recovery check needs no intermediate conversion.